

Rapport Final

SAÉ N°3 : ROBOT SUIVEUR DE LIGNE

Du 21 Mai au 7 Juin 2024

BUTEZ VICTOR, SAMAHRI JIBRIL & CAUREL KILYAN

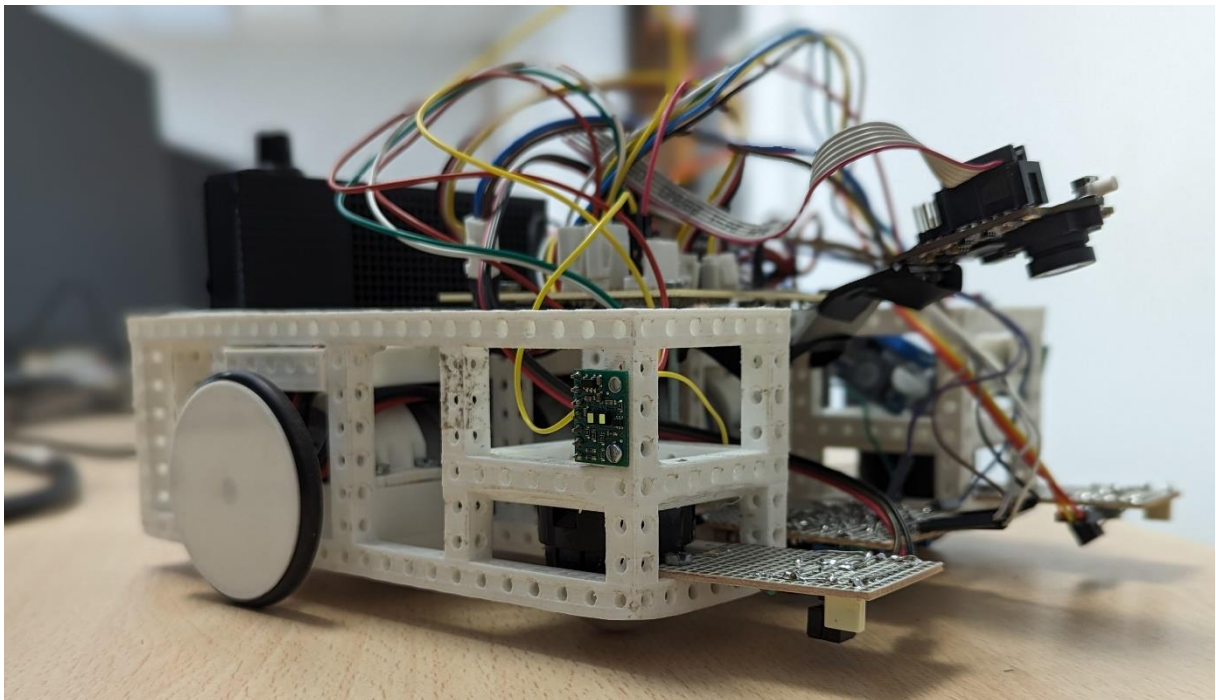
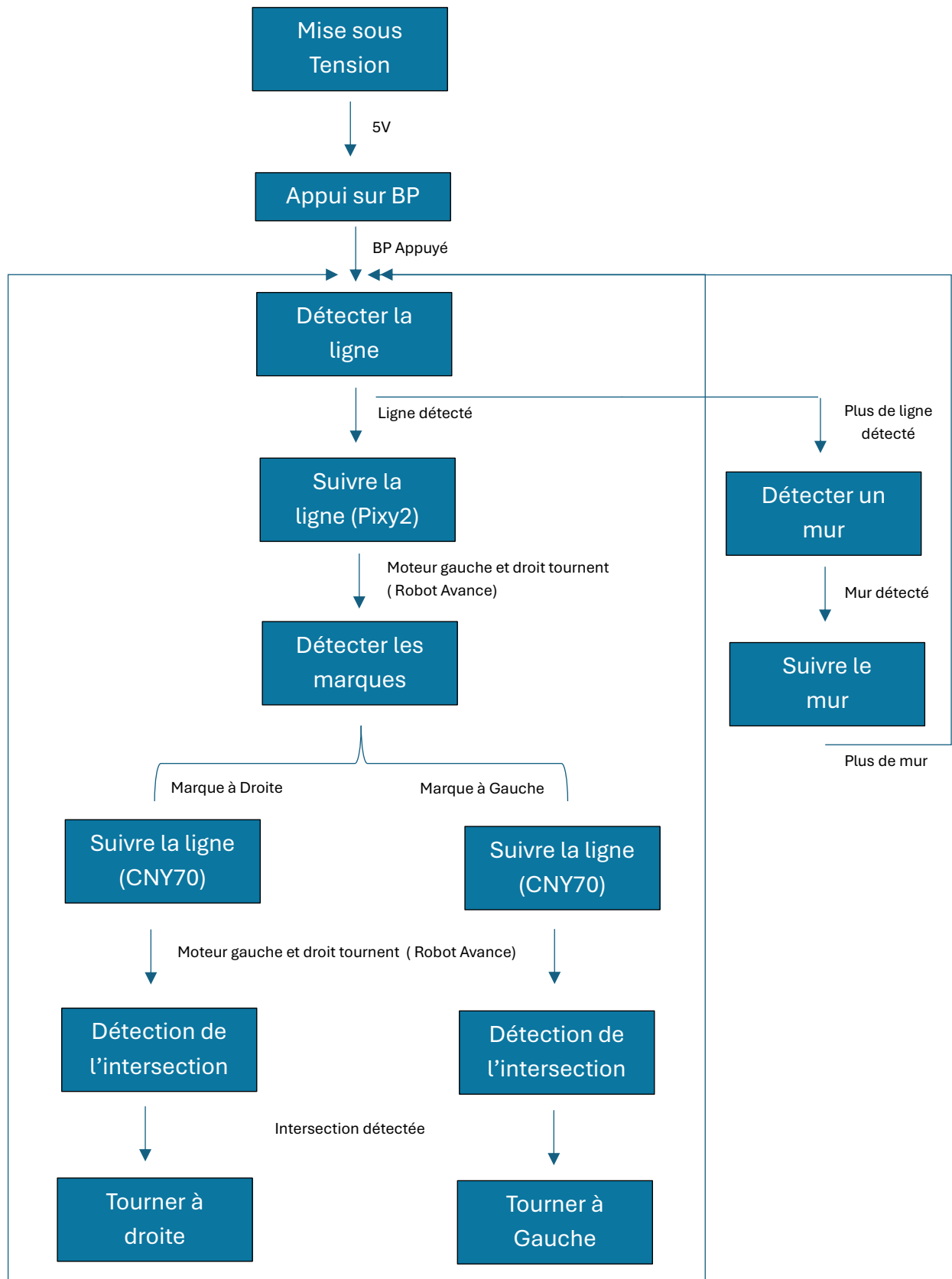


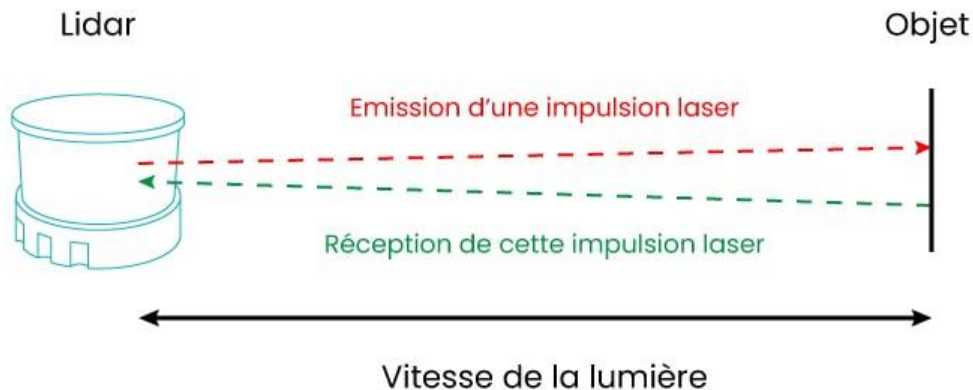
Diagramme d'état :



Définitions des différents Capteurs ainsi que leur utilisation :

LiDAR et Capteur de Distance VL53L1X

Le LiDAR (Light Detection and Ranging) est une technologie de télédétection qui utilise des lasers pour mesurer les distances. Le principe de fonctionnement du LiDAR repose sur l'émission d'une impulsion laser vers une cible et la mesure du temps que met cette impulsion à revenir au capteur après avoir été réfléchi par la cible. Ce temps de vol (Time of Flight, TOF) permet de calculer avec précision la distance entre le capteur et la cible.



Le VL53L1X contient une source laser invisible très compacte et un récepteur adapté. Il fonctionne selon le principe du "Time Of Flight" (temps de vol ou TOF), qui correspond au temps que la source lumineuse a pris pour rebondir sur un objet et revenir vers le récepteur. On appelle souvent ces capteurs des Mini Lidar. Le VL53L1X permet ainsi d'avoir la précision d'un Lidar avec une distance maximale de détection de 4m.

Nous avons utilisé le capteur de distance VL53L1X pour permettre au robot de suivre un mur à une distance de 150 mm. De plus, le robot devait faire tomber la première barre et pas la deuxième. Si le robot détecte un mur à droite alors il suit le mur à droite et si il ne détecte plus de mur à droite il va à gauche et dès qu'il a un mur à gauche il suit ce mur jusqu'à retrouver un mur à droite.

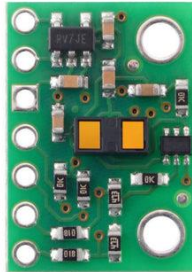
Nous avons rencontré quelques problèmes au niveau des connexions des capteurs LiDAR on a donc fait des câbles Molex pour les connecter sur le Shield Nano et après quelques tentatives on a réussi à faire fonctionner les capteurs VL53L1X ensemble.

Capteur de Distance VL53L1X

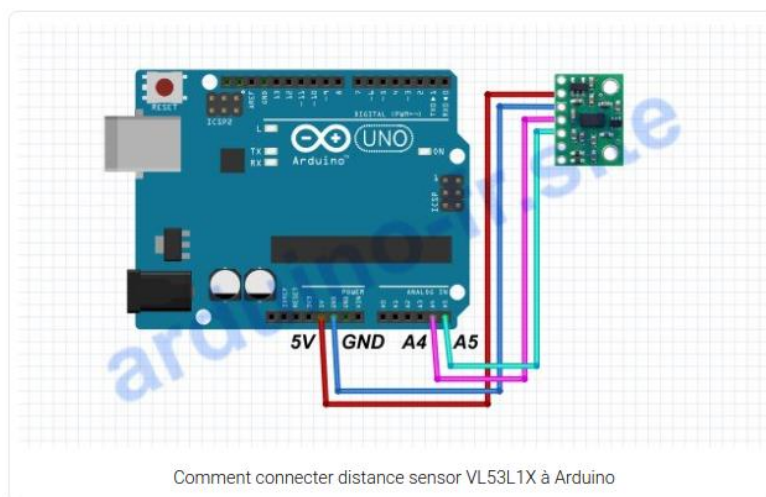
Nous n'utiliserons pas le pin VDD du VL53L1X.



VDD (2.8V out)
VIN (2.6–5.5V)
GND
SDA
SCL
XSHUT
GPIO1



On contrôlera donc les capteurs LiDAR en i2C.



Comment connecter distance sensor VL53L1X à Arduino

VL53LOX / VL53L1X	Arduino Uno	Arduino Nano	Arduino Mega
GND	GND	GND	GND
VDD	5V	5V	5V
SDA	A4	A4	20
SCL	A5	A5	21

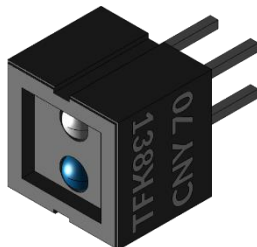
Caméra Pixy2 :

La caméra Pixy2 est une caméra compatible avec le logiciel Arduino et Raspberry Pi. Elle peut être configurée à l'aide du logiciel Pixymon afin de définir ce que l'on veut repérer et avec quelle tolérance. Elle identifie les couleurs ou les lignes que l'on a paramétré avant de réaliser un suivi de l'objet par exemple. Lors de cette SAE, il faut suivre une ligne alors nous configurons la caméra en mode détection de ligne qu'elle va représenter sous forme de vecteur. Nous pourrions alors récupérer des données telles que les coordonnées de l'origine ou de la fin du vecteur en utilisant un logiciel de programmation. Cette caméra peut communiquer par plusieurs moyens (UART, SPI, I2C, USB, digital ou analogique). Ici nous avons sélectionné la voie la plus rapide.

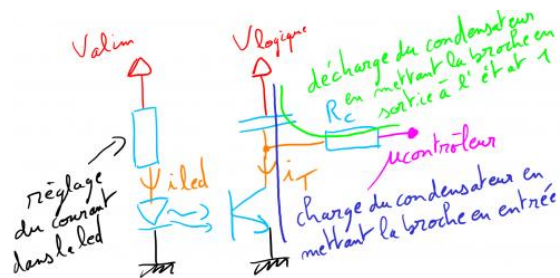
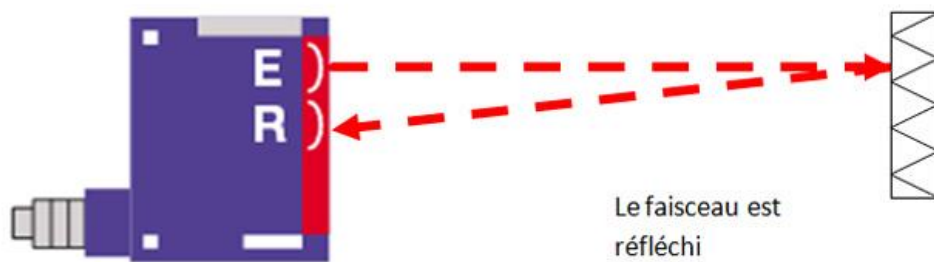


PWM G	PWM D	DIR G	DIR D
PB1	PB2	PD7	PD4

Capteur Photoréfectifs CNY70

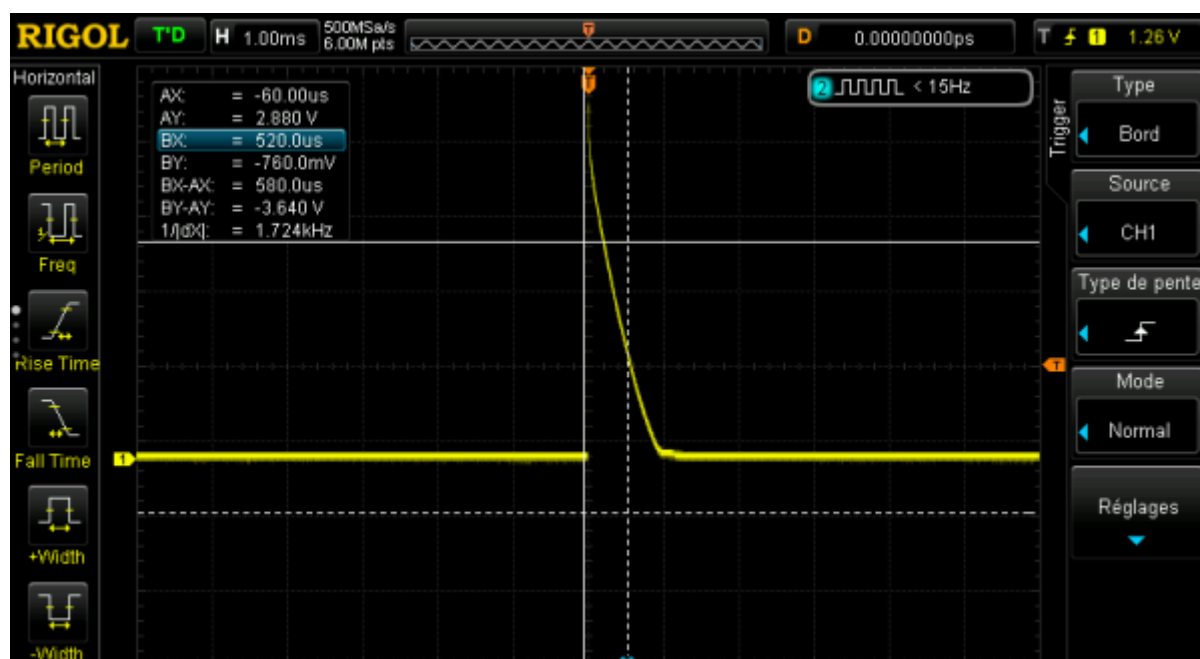


Le CNY70 est un capteur réfléchissant qui contient un émetteur infrarouge et un phototransistor dans un boîtier au plomb qui bloque la lumière visible

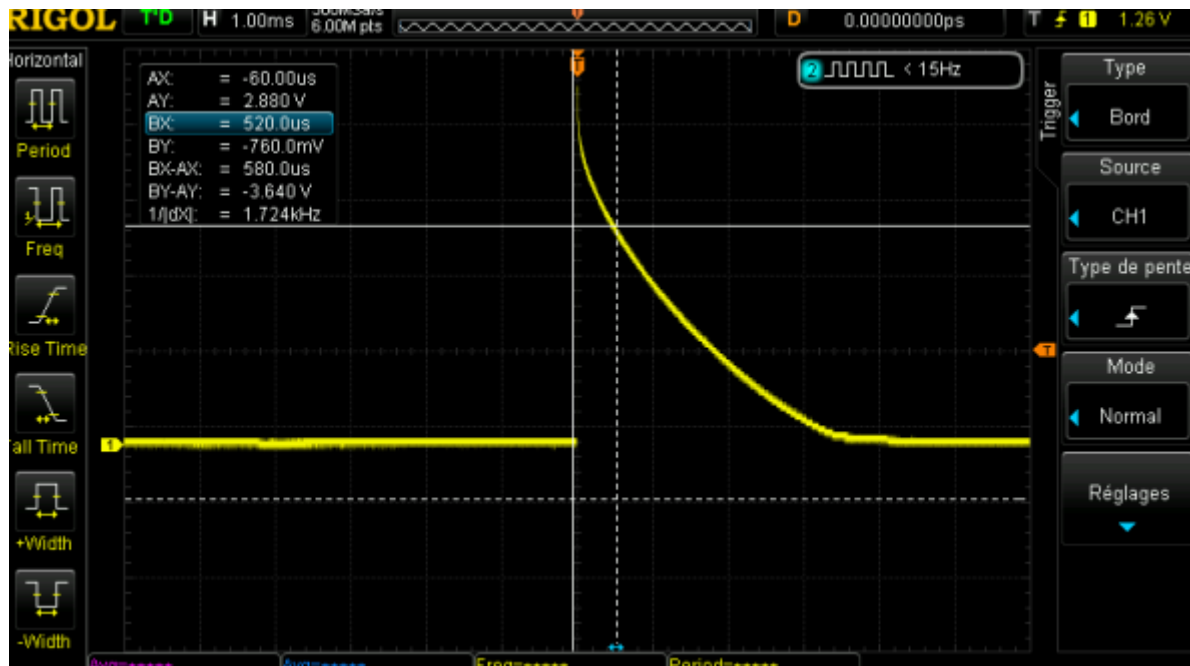


On a choisi ce montage car même si plus complexe à comprendre il permet d'avoir une meilleure efficacité pour la partie capteur photoréfractif du robot. J'ai donc calculé la valeur de la résistance en amont de la led (la valeur du condensateur et de la résistance R_c étant donné 2.2 nF et 220 ohms) on veut un courant de 20mA sur la led et une tension de 1.25V au bornes de celle ci donc en effectuant une loi d'Ohms et des mailles pour obtenir une résistance qui correspond à 220 ohms dans la série E 12.

Courbe de décharges des condensateurs des capteur photo-réfléctifs.



Pour le sol blanc la décharge s'effectue rapidement.



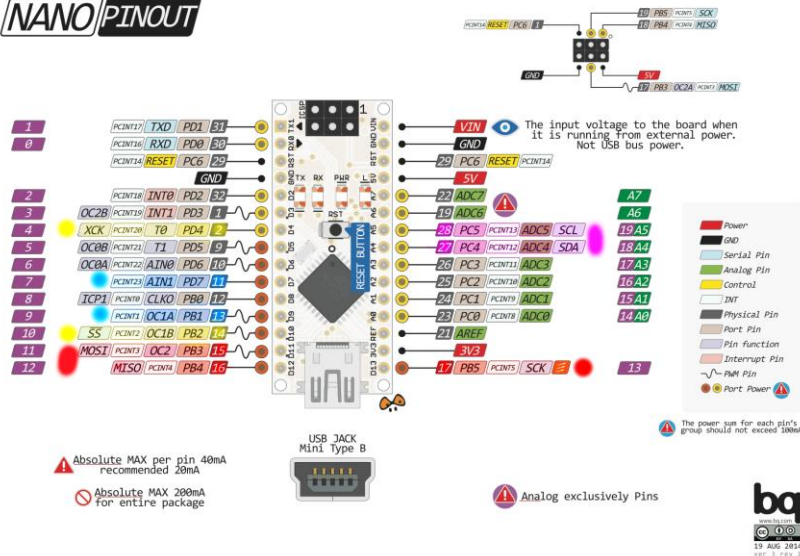
Pour le sol noir, on peut voir que la décharge est beaucoup plus lente.

Les capteurs Photoréfectifs CNY70 avaient pour utilité de suivre la ligne et de détecter les marques à gauche et à droite. Si le robot détecte une marque à gauche alors le robot tourne à gauche à la prochaine intersection, si le robot détecte une marque à droite alors il tourne à droite à la prochaine intersection et si il détecte une marque à gauche et à droite en même temps alors le robot accélère.

Capteur Gauche	Capteur MG	Capteur M	Capteur MD	Capteur Droit
PC0	PC1	PC2	PC3	PC4
A0	A1	A2	A3	A4

PB3 (MOSI)	Pixy
PB4 (MISO)	
PB5 (SCK)	
PB1 (PWM)	Moteur Gauche
PD7 (DIR)	
PB2 (PWM)	Moteur Droit
PD4 (DIR)	
PC4	I2C
PC5	
PB0	Bouton
PD6	Lidar Gauche
PD5	Lidar Obstacle
PD3	Lidar Droit
PD2	INT GPIO
+ / PC0	Signalement Droit
A1 / PC1	Signalement Gauche

NANO PINOUT



Problèmes rencontrés :

Nous avons rencontré quelques problèmes au niveau des connexions des capteurs LIDAR. Nous avons donc fabriqué des câbles Molex pour les connecter sur le Shield Nano et, après quelques tentatives, nous avons réussi à faire fonctionner les capteurs VL53L1X ensemble.

Nous avons également rencontré des problèmes comme le manque de broches sur le Shield Nano pour pouvoir l'alimenter, ce qui nous a contraints à fabriquer une connexion Molex externe au Shield.

Nous avons aussi eu des problèmes avec les Shield pour les CNY70 (détection de marque et ceux du suivi de ligne) au niveau de la soudure. Nous avons donc dû refaire les capteurs de détection de marques et ceux pour le suivi de ligne sur des plaques pastillées.

Améliorations

- Aligner les CNY70 pour le suivi de ligne et détection des marques
- Revoir l'optimisation du temps
- Plus de communication entre les membres du groupe

Programme Final – Parcours Violet (Victor):

Tronc

```
#include "header.h"
/* définition des corrections extrême et centre */
#define ke 200
#define kc 50

int main()
{

uint16_t vtMG = 0;      // vt initiale du moteur gauche
uint16_t vtMD = 0;      // vt initiale du moteur droit
uint16_t correction = 30; // Valeur de l'incrément etatCNY ==
0b00001100 etatCNY == 0b00000110
uint16_t vt = 500;      // Valeur de l'incrément

initFonctionsTempsArduino();
sei();
Serial.begin(115200);    // Moniteur série à 115200 Baud
Wire.begin();
pixy.init();             // On initialise la caméra Pixy
pixy.changeProg("line"); // On met la caméra Pixy en mode ligne
initMli();
initLidar();

while (1)
{
    static uint32_t triggerTime;
    uint32_t currentTime=millis();
    uint8_t etatCNY = readSensorsLine();
    uint8_t etatMark = readSensorsMark();
    uint8_t capteurdroit = etatMark & (1 << PINC0);
    uint8_t capteurgauche = etatMark & (1 << PINC1);
    switch (etapeActive)
    {
        case etapeBP:
        {
            Serial.println(capteurgauche);
            OCR1A = 0; // On arrête le moteur gauche
            OCR1B = 0; // On arrête le moteur droit
            bool etatBP = PINB & (1 << PB0);
            if (etatBP) etapeActive = etapeSuivi;
            break;
        }
    }
}
```

```

case etapeSuivi:
{
    Serial.println("suivi");
    float wr = (ke * (((etatCNY & 0b00010000)>>4) - ((etatCNY &
0b00000001)>>0)) + kc * (((etatCNY & 0b00001000)>>3) - ((etatCNY &
0b00000010)>>1)));
    float vRoueGauche = vt + wr;
    float vRoueDroite = vt - wr;
    speedmotor(vRoueGauche,vRoueDroite);
    if ((capteurdroit) ==
1) //&&(etatCNY==(0b00001100)|(0b00000110)))
    {
        etapeActive = suiviDroit;
    }
    if ((capteurgauche) == 2) etapeActive = suiviGauche;
    else if (etatCNY == 0) etapeActive = LidarDroit;
    break;
}

case suiviDroit:
{
    Serial.println("suiviDroit");
    float wr = (ke * (((etatCNY & 0b00010000)>>4) - ((etatCNY &
0b00000001)>>0)) + kc * (((etatCNY & 0b00001000)>>3) - ((etatCNY &
0b00000010)>>1)));
    float vRoueGauche = vt + wr;
    float vRoueDroite = vt - wr;
    speedmotor(vRoueGauche,vRoueDroite);
    //if ((etatCNY== (
0b000011100)|(0b00001110)|(0b00000111)|(0b00011110)|(0b00001111)|(0b000
11111) ) )
    if ( (etatCNY== (0b00011100)) || (etatCNY== (0b00001110)) ||
(etatCNY== (0b00000111)) || (etatCNY== (0b00011110)) || (etatCNY==
(0b00001111)) || (etatCNY== (0b00011111)) )
    {
        etapeActive = etapeIntersectionD;
        triggerTime=currentTime;
    }

    break;
}
case etapeIntersectionD :

```

```

{
    Serial.println("IntersectionD");
    speedmotor(500, 0);
    if (currentTime >= (triggerTime + 1200 ))
    {
        speedmotor(500, 0);
        etapeActive = etapeTournerDJ;
        //triggerTime=currentTime;
    }
    break;
}

case etapeTournerDJ :
{
    Serial.println("TournerDJ");
    while ((etatCNY) == (0b00000000)) //&& (currentTime >=
(triggerTime + 1000 ))
    {
        speedmotor(500, 50);
        Serial.println(etatCNY, BIN);
    }
    etapeActive=etapeSuivi;
    break;
}
case suiviGauche:
{
    Serial.println("suiviGauche");
    float wr = (ke * (((etatCNY & 0b00010000)>>4) - ((etatCNY &
0b00000001)>>0)) + kc * (((etatCNY & 0b00001000)>>3) - ((etatCNY &
0b00000010)>>1)));
    float vRoueGauche = vt + wr;
    float vRoueDroite = vt - wr;
    speedmotor(vRoueGauche,vRoueDroite);
    //if ((etatCNY== (
(0b00011100)|(0b00001110)|(0b00000111)|(0b00011110)|(0b00001111)|(0b000
11111) ) ) )
    if ( (etatCNY== (0b00011100)) || (etatCNY== (0b00001110)) ||
(etatCNY== (0b00000111)) || (etatCNY== (0b00011110)) || (etatCNY==
(0b00001111)) || (etatCNY== (0b00011111)) )
    {
        etapeActive = etapeIntersectionG;
        triggerTime=currentTime;
    }

    break;
}
case etapeIntersectionG :
{
    Serial.println("IntersectionG");
    speedmotor(50, 500);
    if (currentTime >= (triggerTime + 1500 ))

```

```

    {
        speedmotor(0, 500);
        etapeActive = etapeTournerGJ;
        //triggerTime=currentTime;
    }
    break;
}

case etapeTournerGJ :
{
    Serial.println("TournerGJ");
    while ((etatCNY) == (0b00000000)) //&& (currentTime >=
(triggerTime + 1000 ))
    {
        speedmotor(50, 500);
        Serial.println(etatCNY, BIN);
    }
    etapeActive=etapeSuivi;
    break;
}

case LidarDroit :
{
    capteurGauche = sensors[0].read();
    capteurDroit = sensors[1].read();
    _delay_ms(100);
    int16_t erreurDroite = capteurDroit - distanceConsigne; // Calcul
de l'erreur à droite
    int16_t erreurGauche = capteurGauche - distanceConsigne; //
Calcul de l'erreur à gauche
    double Kp = 0.03; //
Coefficient proportionnel (temps de reponse)
    double Ki = 1.5; //
Coefficient intégral (temps de derivation)

    // Variables pour l'action intégrale
    double integralDroite = 0;
    double integralGauche = 0;

    // Déterminer la présence des murs
    bool murADroite = (capteurDroit < 400);
    bool murAGauche = (capteurGauche < 400);

```

```

    // Mise à jour de l'action intégrale
    integralDroite += erreurDroite;
    integralGauche += erreurGauche;

    // Coefficient proportionnel
    int16_t wrDroite = Kp * erreurDroite + Ki * integralDroite;
    int16_t wrGauche = Kp * erreurGauche + Ki * integralGauche;

    if (murADroite)
    {
        // Si le mur à droite est détecté
        vtMG = vt + wrDroite;
        vtMD = vt - wrDroite;
        speedmotor(vtMG, vtMD);
    }
    else
    {
        vtMG=2000;
        vtMD=200;
    }
    // On modifie l'étape active pour la prochaine boucle
    etapeActive = etapeSuivi;
    _delay_ms(50); // Petite pause pour éviter de surcharger la
boucle
    }
}
}
}

```

I2C.ino

```
uint8_t readI2cReg(uint8_t targetAddress, uint8_t regAddress) {
    uint8_t regValue = 0;
    Wire.beginTransmission(targetAddress); // start transmitting
    Wire.write(regAddress);                // sends @ registre
    Wire.endTransmission();                // stop transmitting
    Wire.requestFrom(targetAddress, (uint8_t)1);
    while (Wire.available()) regValue = Wire.read();
    return regValue;
}

void writeI2cReg(uint8_t targetAddress, uint8_t regAddress,
uint8_t regValue) {
    Wire.beginTransmission(targetAddress); // start transmitting
    Wire.write(regAddress);                // sends @ registre
    Wire.write(regValue);                  // sends value registre
    Wire.endTransmission();                // stop transmitting
}
```

header.h

```
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <Pixy2.h>
#include <Wire.h>
#include <VL53L1X.h>

#define IODIR 0x00
#define IPOL 0x01
#define GPINTEN 0x02
#define DEFVAL 0x03
#define INTCON 0x04
#define IOCON 0x05
#define GPPU 0x06
#define INTF 0x07
#define INTCAP 0x08
#define GPIO 0x09
#define OLAT 0x0A

uint8_t readI2cReg(uint8_t targetAddress, uint8_t regAddress);
void writeI2cReg(uint8_t targetAddress, uint8_t regAddress, uint8_t
regValue);
// Fonctions pour contrôler les moteurs du robot
void speedmotor(uint16_t vtMG, uint16_t vtMD);

const uint8_t targetAdress = 0x20;

Pixy2 pixy;

// Nombre de capteurs dans le système
```

```

const uint8_t sensorCount = 2;

// Pins Arduino connectées aux pins XSHUT de chaque capteur
const uint8_t xshutPins[sensorCount] = {3, 5};

VL53L1X sensors[sensorCount];

// Variables pour les lectures des capteurs
uint16_t capteurGauche;
uint16_t capteurDroit;

// Pins controle direction moteurs
const uint8_t dirPin1 = 7; // moteur gauche
const uint8_t dirPin2 = 4; // moteur droite

// Distance consigne
const uint16_t distanceConsigne = 200;

// Indiquer etat marche robot
bool robotEnMarche = false;

uint16_t x_consigne = 40; // Milieu de l'écran
uint16_t avancer = 1000; // vt de base
uint16_t K = 20; // Coefficient courbe
uint16_t C = 20; // Coefficient position actuelle

enum state { etapeBP, etapeSuivi, etapeAttente, suiviDroit,
suiviGauche, etapeDRS, etapeIntersectionG, etapeIntersectionD,
etapeTournerGJ, etapeTournerDJ, LidarDroit};
state etapeSuivante = etapeBP;
state etapeActive = etapeBP;

```

initFonctionsTempsArduino.ino

```

void initFonctionsTempsArduino()
{
    sei();
    // marche pour 328p et 2560 et autres ...
    // à adapter suivant le µc
    // cf https://github.com/arduino/ArduinoCore-avr/blob/master/cores/arduino/wiring.c
    TCCR0A=(1<<WGM01) | (1<<WGM00);
    TCCR0B=(1<<CS01) | (1<<CS00);
    TIMSK0=(1<<TOIE0);
}

```



```
}
```

initMli.ino

```
void initMli(){
  // Configuration de la MLI
  DDRB |= (1 << PB1) | (1 << PB2);

  TCCR1A |= (1 << WGM11);
  // valeur du prédiviseur
  TCCR1B |= (1 << WGM13) | (1 << WGM12) | (1 << CS11);
  // générer le signal MLI sur la broche OC1A ou OC1B
  TCCR1A |= (1 << COM1A1) | (1 << COM1B1);
  // valeur du rapport cyclique
  ICR1 = 4000;
}
```

readSensorsLine.ino

```
uint8_t readSensorsLine()
{
  writeI2cReg(targetAdress, IODIR, 0b00000000);
  writeI2cReg(targetAdress, GPIO, 0b00011111);
  _delay_us(20);
  writeI2cReg(targetAdress, IODIR, 0b00011111);
  return readI2cReg(targetAdress, GPIO) & 0b00011111;
}
```

readSensorsMark.ino

```
uint8_t readSensorsMark()
{
  DDRC |= (1 << PC0) | (1 << PC1);
  PORTC |= (1 << PC0) | (1 << PC1);
  _delay_us(20);
  DDRC &= ~(1 << PC0) | (1 << PC1);
  _delay_us(500);
  return PINC & 0b00000011;
}
```

speedmotor.ino

```
// Fonctions pour contrôler les moteurs du robot
```

```

void speedmotor(uint16_t vtMG, uint16_t vtMD)
{
    /* direction Cytron */
    DDRD |= (1 << PD4) | (1 << PD7);
    OCR1A = vtMG;
    OCR1B = vtMD;
}

```

initLidar.ino

```

void initLidar()
{
    Serial.begin(115200);
    Wire.begin();
    Wire.setClock(400000); // Utiliser une fréquence I2C de 400 kHz

    // Désactiver/réinitialiser tous les capteurs en mettant leurs pins
    XSHUT à bas
    for (uint8_t i = 0; i < sensorCount; i++)
    {
        pinMode(xshutPins[i], OUTPUT);
        digitalWrite(xshutPins[i], LOW);
    }

    // Activer, initialiser et démarrer chaque capteur un par un
    for (uint8_t i = 0; i < sensorCount; i++)
    {
        pinMode(xshutPins[i], INPUT);
        delay(10);
        sensors[i].setTimeout(500);
        if (!sensors[i].init())
        {
            Serial.print("Échec détection et initialisation du capteur ");
            Serial.println(i);
            //while (1);
        }

        // Chaque capteur doit avoir une adresse unique différente de la
        valeur par défaut de 0x29
        sensors[i].setAddress(0x2A + i);
        sensors[i].startContinuous(50);
    }
}

```

Programme Final – Parcours Vert (Jibril):

SUIVEUR DE LIGNE
AVEC GPIO + VITESSE
MOYENNE + capteur doit
et gauche:

```
#include <avr/io.h>
#include <util/delay.h>
#include <Arduino.h> // Assurez-vous d'avoir l'IDE Arduino pour Serial
#include <Wire.h>
uint8_t etatPort;
const uint8_t targetAddress = 0x20;
#define IODIR 0x00
#define IPOL 0x01
#define GPINTEN 0x02
#define DEFVAL 0x03
#define INTCON 0x04
#define IOCON 0x05
#define GPPU 0x06
#define INTF 0x07
#define INTCAP 0x08
#define GPIO 0x09
#define OLAT 0x0A

uint8_t readI2cReg(uint8_t targetAddress, uint8_t regAddress) {
    uint8_t regValue = 0;
    Wire.beginTransmission(targetAddress); // start transmitting
    Wire.write(regAddress);                // sends @ registre
    Wire.endTransmission();                // stop transmitting
    Wire.requestFrom(targetAddress, (uint8_t)1);
    while (Wire.available()) regValue = Wire.read();
    return regValue;
}

void writeI2cReg(uint8_t targetAddress, uint8_t regAddress, uint8_t
regValue) {
    Wire.beginTransmission(targetAddress); // start transmitting
    Wire.write(regAddress);                // sends @ registre
    Wire.write(regValue);                  // sends value registre
    Wire.endTransmission();                // stop transmitting
}

// Fonctions pour contrôler les moteurs du robot
void speedmotor(uint16_t vitesseMG, uint16_t vitesseMD) {
    OCR1A = vitesseMG;
    OCR1B = vitesseMD;
```

```
}
```

```
int main() {
Wire.begin();
Serial.begin(115200);
Wire.setClock(100000ul);
// Configure PC0 à PC4 comme entrées
//writeI2cReg(targetAdress, IODIR, 0b11111111);
// Configuration de la MLI
DDRB |= (1 << PB1) | (1 << PB2);
DDRD |= (1 << PD4) | (1 << PD7); //PD4=gauche
// timer1 en mode 14
TCCR1A |= (1 << WGM11);
// valeur du prédiviseur
TCCR1B |= (1 << WGM13) | (1 << WGM12) | (1 << CS11);
// générer le signal MLI sur la broche OC1A ou OC1B
TCCR1A |= (1 << COM1A1) | (1 << COM1B1);
// valeur du rapport cyclique
ICR1 = 4000;
uint16_t vitesseMG = 0; // Vitesse initiale du moteur gauche
uint16_t vitesseMD = 0; // Vitesse initiale du moteur droit
uint16_t maxSpeed = 4000; // Vitesse maximale (rapport cyclique maximal)
uint16_t k = 2; // Valeur de l'incrément
uint16_t correction = 70; // Valeur de l'incrément etatPort ==
0b00001100 etatPort == 0b00000110
uint16_t vitesse = 1100; // Valeur de l'incrément
sei();

while (1) {
writeI2cReg(targetAdress, IODIR, 0b00000000);
writeI2cReg(targetAdress, GPIO, 0b00011111);
_delay_us(20);
writeI2cReg(targetAdress, IODIR, 0b00011111);
etatPort = readI2cReg(targetAdress, GPIO);

DDRC |= (1<<PC0)|(1<<PC1) ;
PORTC |= (1<<PC0)|(1<<PC1) ;
_delay_us(20);
DDRC &= ~((1<<PC0)|(1<<PC1)) ;
_delay_us(500);
uint16_t capteurdroit = PINC & (1 << PINC0);
uint16_t capteurgauche = PINC & (1 << PINC1);
/*
Serial.print(" 5 capteurs ");
Serial.println(etatPort, BIN);
```

```

Serial.print("capteur droit ");
Serial.println(capteurdroit, BIN);
/*
Serial.print("capteur gauche ");
*/
Serial.println(capteurgauche, BIN);

if (etatPort == 0b00000100) {
    // Si le capteur central est sur la ligne noire
    speedmotor(vitesse, vitesse);

} else if (etatPort == 0b00001100) {
    vitesseMD = vitesse-correction;
    vitesseMG = vitesse;
    speedmotor(vitesseMG, vitesseMD);

} else if (etatPort == 0b00000110) {
    vitesseMD = vitesse ;
    vitesseMG = vitesse- (correction);
    speedmotor(vitesseMG, vitesseMD);

}

else if (etatPort == 0b00000011) {

    vitesseMD = vitesse + ( correction);
    vitesseMG = vitesse - (6*correction);
    speedmotor(vitesseMG, vitesseMD);

} else if (etatPort == 0b00011000) {
    // Si aucun capteur n'est sur la ligne noire, arrêter (ou continuer
    tout droit)

```

```

    vitesseMD = vitesse - (6*correction);
    vitesseMG = vitesse + (correction);
    speedmotor(vitesseMG, vitesseMD);
} else if (etatPort == 0b00010000) {
    // Si aucun capteur n'est sur la ligne noire, arrêter (ou continuer
    tout droit)

    vitesseMD = vitesse-(8 * correction);
    vitesseMG = vitesse + (3 * correction);
    speedmotor(vitesseMG, vitesseMD);

} else if (etatPort == 0b00000001) {
    // Si aucun capteur n'est sur la ligne noire, arrêter (ou continuer
    tout droit)

    vitesseMD = vitesse + (3 * correction);
    vitesseMG = vitesse - (8 * correction);
    speedmotor(vitesseMG, vitesseMD);
    PORTD &= ~(1 << PD4) | (1 << PD7);

}
else if (etatPort == 0b00000000) {
    // Si aucun capteur n'est sur la ligne noire, arrêter (ou continuer
    tout droit)

    speedmotor(0, 0);
}
return 0;
}

```

Partie pixy:

```

    if (pixy.line.numVectors) // Si vecteur détecté
    {

```

```

    int16_t x_origine = pixy.line.vectors[0].m_x0; // Coordonnées X
à l'origine du vecteur
    int16_t x_end = pixy.line.vectors[0].m_x1;      // Coordonnées X
à la fin du vecteur
    int16_t x_diff = x_end - x_origine;             // Différence
entre la fin et le début du vecteur (axe X) pour connaître la
trajectoire à suivre
    int16_t Wr = K * x_diff + C * (x_origine - x_consigne); //
Vitesse angulaire en fonction de la position actuelle et la trajectoire
à suivre (axe X)
    OCR1A = avancer + Wr; // MG
    OCR1B = avancer - Wr; // MD
    PORTD &= ~(1 << PD4); // Sens de rotation du moteur droit
    PORTD &= ~(1 << PD7); // Sens de rotation du moteur gauche

```

Programme Final – Parcours Bleu (Kilyan):

Tronc :

```
#define F_CPU 8000000UL // Fréquence du CPU
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <stdint.h>
#include <Pixy2.h>
#include <Wire.h>
#include <VL53L1X.h>

Pixy2 pixy;
uint16_t x_consigne = 40; // Milieu de l'écran
uint16_t avancer = 1000; // Vitesse de base
uint16_t K = 20; // Coefficient courbe
uint16_t C = 20; // Coefficient position actuelle

enum state { etapeBP, etapeSuivi, etapeSuiviMur };
state etapeSuivante = etapeBP;
state etapeActive = etapeBP;

int main() {
    Serial.begin(115200); // Moniteur série à 115200 Baud
    pixy.init(); // On initialise la caméra Pixy
    initMli();
    pixy.changeProg("line"); // On met la caméra Pixy en mode ligne

    while (1) {
        pixy.line.getMainFeatures(); // On active les fonctions principales de
        la caméra Pixy

        switch (etapeActive) {
            case etapeBP: {
                bool etatBP = PINB & (1 << PB0);
                if (etatBP) etapeActive = etapeSuivi;
                break;
            }
            case etapeSuivi: {
                SuiviLigne();
                break;
            }
        }
    }
}
```



```

    }
    case etapeSuiviMur: {
        SuiviMur();
        break;
    }
}
}
}
}

```

SuiviLigne.ino (Pixy)

```

void SuiviLigne() {
    if (pixy.line.numVectors) { // Si vecteur détecté
        int16_t x_origine = pixy.line.vectors[0].m_x0; // Coordonnées X à
l'origine du vecteur
        int16_t x_end = pixy.line.vectors[0].m_x1; // Coordonnées X à la
fin du vecteur
        int16_t x_diff = x_end - x_origine; // Différence entre la
fin et le début du vecteur (axe X) pour connaître la trajectoire à suivre
        int16_t Wr = K * x_diff + C * (x_origine - x_consigne); // Vitesse
angulaire en fonction de la position actuelle et la trajectoire à suivre (axe
X)

        Serial.print("x_diff: ");
        Serial.println(x_diff);
        Serial.print("x: ");
        Serial.println(x_origine - x_consigne);

        OCR1A = avancer + Wr; // MG
        OCR1B = avancer - Wr; // MD
        PORTD &= ~(1 << PD4); // Sens de rotation du moteur droit
        PORTD &= ~(1 << PD7); // Sens de rotation du moteur gauche
    } else { // Pas de vecteur détecté
        OCR1A = 0; // On arrête le moteur gauche
        OCR1B = 0; // On arrête le moteur droit
        etapeActive = etapeBP;
    }
}
}

```

SuiviMur.ino

```

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <Wire.h>
#include <VL53L1X.h>

```

```

// Nombre de capteurs dans le système
const uint8_t sensorCount = 2;

// Pins Arduino connectées aux pins XSHUT de chaque capteur
const uint8_t xshutPins[sensorCount] = {3, 5};

VL53L1X sensors[sensorCount];

// Variables pour les lectures des capteurs
uint16_t capteurGauche;
uint16_t capteurDroit;

// Pins controle direction moteurs
const uint8_t dirPin1 = 7; // moteur gauche
const uint8_t dirPin2 = 4; // moteur droite

// Distance consigne
const uint16_t distanceConsigne = 200;

void initFonctionsTempsArduino() {
    sei(); // Activer les interruptions globales
    // Configurer le Timer0 (utilisé pour les fonctions de timing générales dans
    Arduino)
    TCCR0A = (1 << WGM01) | (1 << WGM00);
    TCCR0B = (1 << CS01) | (1 << CS00);
    TIMSK0 = (1 << TOIE0);
}

void initLidar() {
    Serial.begin(115200);
    Wire.begin();
    Wire.setClock(400000); // Utiliser une fréquence I2C de 400 kHz

    // Désactiver/réinitialiser tous les capteurs en mettant leurs pins XSHUT à
    bas
    for (uint8_t i = 0; i < sensorCount; i++) {
        pinMode(xshutPins[i], OUTPUT);
        digitalWrite(xshutPins[i], LOW);
    }

    // Activer, initialiser et démarrer chaque capteur un par un
    for (uint8_t i = 0; i < sensorCount; i++) {
        pinMode(xshutPins[i], INPUT);
        delay(10);
        sensors[i].setTimeout(500);
    }
}

```

```

    if (!sensors[i].init()) {
        Serial.print("Échec détection et initialisation du capteur ");
        Serial.println(i);
        //while (1);
    }

    // Chaque capteur doit avoir une adresse unique différente de la valeur
    par défaut de 0x29
    sensors[i].setAddress(0x2A + i);
    sensors[i].startContinuous(50);
}
}

void speedmotor(int16_t vitesseGauche, int16_t vitesseDroite) {
    OCR1A = vitesseGauche;
    OCR1B = vitesseDroite;
}

void SuiviMur() {
    initFonctionsTempsArduino();
    initLidar();
    initMli();
    // initButton(); // Assuming you have a function to initialize button which
    was mentioned but not defined in the original code

    while (1) {
        // Lire les distances des capteurs
        capteurGauche = sensors[0].read();
        capteurDroit = sensors[1].read();

        Serial.print("Capteur Gauche: ");
        Serial.print(capteurGauche);
        if (sensors[0].timeoutOccurred()) {
            Serial.print(" TIMEOUT");
        }
        Serial.print("\t");

        Serial.print("Capteur Droit: ");
        Serial.print(capteurDroit);
        if (sensors[1].timeoutOccurred()) {
            Serial.print(" TIMEOUT");
        }
        Serial.print("\t");

        _delay_ms(100);

        Serial.println("Suivi");
    }
}

```

```

    int16_t erreurDroite = capteurDroit - distanceConsigne; // Calcul de
l'erreur à droite
    int16_t erreurGauche = capteurGauche - distanceConsigne; // Calcul de
l'erreur à gauche
    int vt = 500; // la vitesse du
robot
// Coefficient
proportionnel // Coefficients de
contrôle
    double Kp = 0.1; // Coefficient
proportionnel (temps de reponse)
    double Ki = 0.4; // Coefficient
intégral (temps de derivation)

// Variables pour l'action intégrale
double integralDroite = 0;
double integralGauche = 0;

// Déterminer la présence des murs
bool murADroite = (capteurDroit < 300);
bool murAGauche = (capteurGauche < 300);

// Mise à jour de l'action intégrale
integralDroite += erreurDroite;
integralGauche += erreurGauche;

// Coefficient proportionnel
int16_t wrDroite = Kp * erreurDroite + Ki * integralDroite;
int16_t wrGauche = Kp * erreurGauche + Ki * integralGauche;

//variables intermédiaires
int16_t vitesseGauche;
int16_t vitesseDroite;

if (murADroite) {
    // Si le mur à droite est détecté
    vitesseGauche = vt + wrDroite;
    vitesseDroite = vt - wrDroite;
    speedmotor(vitesseGauche, vitesseDroite);
} else if (murAGauche) {
    // Si le mur à gauche est détecté
    vitesseGauche = vt - wrGauche;
    vitesseDroite = vt + wrGauche;
    speedmotor(vitesseGauche, vitesseDroite);
}
    _delay_ms(50); // Petite pause pour éviter de surcharger la boucle
}
}

```

InitMli.ino

```
void initMli() {  
    // Configurer les pins de contrôle des moteurs en sorties  
    DDRB |= (1 << PB1) | (1 << PB2); // motorPin1, motorPin2 (D9, D10)  
    DDRD |= (1 << PD4) | (1 << PD7); // dirPin1, dirPin2 (D3, D4)  
    PORTD &= ~(1 << PD4); // Sens de rotation du moteur droit  
    PORTD &= ~(1 << PD7); // Sens de rotation du moteur gauche  
    TCCR1A |= (1 << WGM11); // Fast PWM  
    TCCR1B |= (1 << WGM13) | (1 << WGM12) | (1 << CS11); // Prédiviseur=8  
    TCCR1A |= (1 << COM1A1); // À 0 quand OCR1A est atteint et 1 quand 0 est  
atteint  
    TCCR1B |= (1 << COM1B1); // À 0 quand OCR1B est atteint et 1 quand 0 est  
atteint  
    // Valeur du rapport cyclique  
    OCR1A = 0;  
    OCR1B = 0;  
    ICR1 = 4000; // Fréquence de la MLI sachant que  $ICR1 = f_{CPU} / (P * MLI)$   
}
```

QUELQUES PHOTOS DU ROBOT :

